Module 7-Generative AI & AI Agenting Assignment

Generative AI for Data Engineering – Assignment

NOTE: -

Kindly create a Gemini Api key using link – https://aistudio.google.com/app/api-keys

The Udemy courses might have asked you to create a OpenAI Api key, but it bears some cost. For learning purposes, Gemini Api key should be fine. Also use low-cost models for lesser token usage.

Example of OpenAI code vs Gemini code (to programmatically interact with respective LLMs) -

OpenAI -

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": """Summarize the following JSON dataset and suggest 2 data quality checks:
{
  'orders': [
    {'id': 1, 'amount': 250, 'status': 'completed'},
    {'id': 2, 'amount': -50, 'status': 'completed'},
    {'id': 3, 'amount': 300, 'status': null}
  ]
}"""}
    ]
)

print(response.choices[0].message.content)
```

Gemini -

```
import google.generativeai as genai

genai.configure(api_key="YOUR_GEMINI_API_KEY")

model = genai.GenerativeModel("gemini-1.5-flash")

response = model.generate_content("""
Summarize the following JSON dataset and suggest 2 data quality checks:
{
  'orders': [
    {'id': 1, 'amount': 250, 'status': 'completed'},
    {'id': 2, 'amount': -50, 'status': 'completed'},
    {'id': 3, 'amount': 300, 'status': null}
  ]
}
""")

print(response.text)
```

Prompt Engineering Fundamentals for Data Engineering:

Prepare and Submit 3 well-structured prompts that you might give to LLMs, as a data engineer. The prompts should include all key components: Context, Role, Task, Constraints, Format, and (where applicable) Examples.

Example Prompt:

Context: I am working with a large fact table (1B+ rows) in a data warehouse and queries are running slowly. <long running query here> **Role:** Act as a senior data warehouse optimization expert.

Task: Optimize the given SQL query for performance. **Constraints:**

- Focus on performance improvements only
- Assume distributed compute environment (like Databricks/Snowflake) **Format:**
- Optimized query Examples**:**

Programmatically Chat with an LLM & Format Output

You are given a dataset of user activity logs. Your task is to:
Write a Python program to interact with an LLM

Send the following input to the LLM:
User activity:

- User A logged in and purchased a laptop worth $1200
- User B logged in but did not make any purchase
- User C purchased a phone worth $800

Ask the LLM to:
Summarize user activity Extract
structured insights Your program must:
-Use Python to call the LLM API
-Pass a well-structured prompt
-Format the output as below JSON

```
{
 "summary": "...",
 "total_users": 3,
 "purchasing_users": 2,
 "total_revenue": 2000,
 "insights": [
  "...",
  "..."
 ]
}
```

Data Generation & Augmentation

Read a csv file in your python program. Then create more similar data by interacting with LLM. Scenarios like this will help you create more data for testing scenarios when actual available data for testing is limited available.

Data Querying

Pass a pdf document or a word document to the gen ai application (basic application to connect programmatically to LLM) that you have created. Now ask questions to LLM, based on the sample pdf/word document, this will help you understand if LLM is reading your pdf/docs and understanding your questions well or not. It might hallucinate or give completely accurate answers. Observe how LLM behaves with different types of questions asked.

Data Extraction

Create the following schema in sqllite3 – customer (customer_id
INTEGER PRIMARY KEY, name TEXT, email TEXT, join_date TEXT)

sales (sale_id INTEGER PRIMARY KEY, customer_id INTEGER,
product TEXT, amount REAL,
sale_date TEXT,
FOREIGN KEY(customer_id) REFERENCES customer(customer_id))

Now pass this in prompt to a gen ai application and also ask llm to generate a valid SQL query by translating natural language to SQL e.g. – (natural language) highest sales amount done by a customer in last 3 days, etc.

Once it generates a SQL query, execute the same on top of db where you have created the above schema and those tables are populated with data.

Finally format & display the answer the obtained by executing LLM returned query.
For example – if sql query returns 0 rows, then your program should return something like – "No results for the query."
Tech stack you may use – OpenAI Api keys, python, SQL alchemy, sqllite3 etc.

More important topics (Theory)

Read topics related to Vector DB, embedding, chunking, RAG systems, mcp servers. Try to understand each of this topic at the fundamental level. These are the starting points in building production grade Gen AI/ Agentic AI systems.

Deliverables:

Deploy the application on the Azure cloud platform, capture screenshots of the successful deployment/submission, and ensure all deployed resources are stopped or deleted after completion.